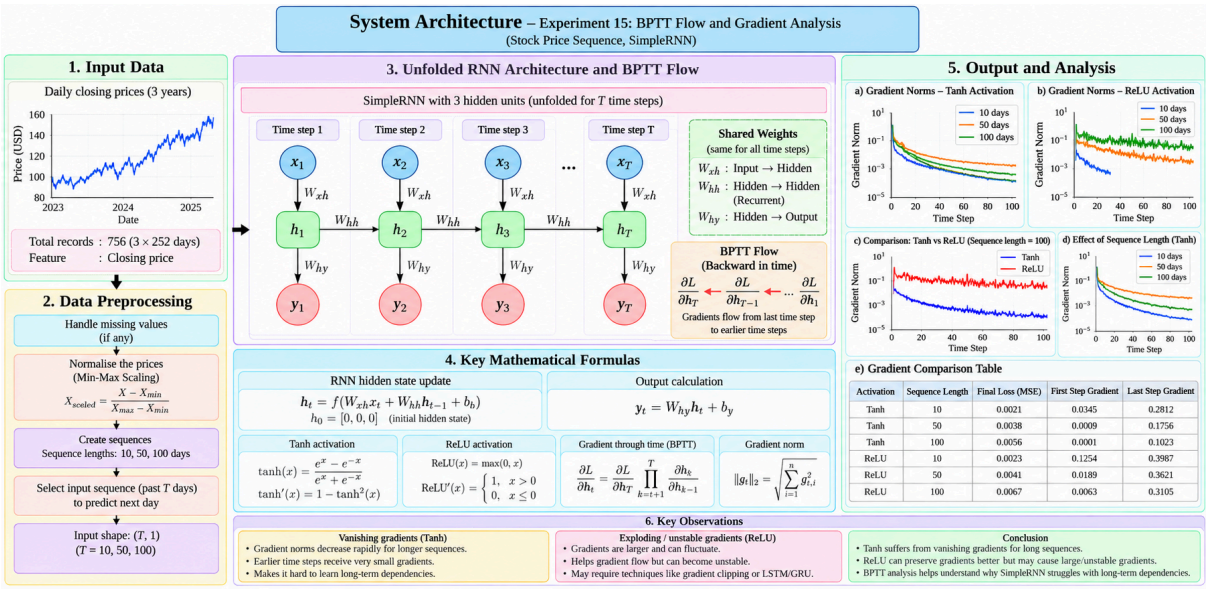
Experiment 15: BPTT Flow and Gradient Analysis

1. Aim

To implement a Simple Recurrent Neural Network (SimpleRNN) and analyse Backpropagation Through Time (BPTT) using TensorFlow and Keras. To calculate gradient magnitudes at different time steps for sequence lengths of 10, 50, and 100 days using `tf.GradientTape`, compare tanh and ReLU activation functions, visualize gradient norms, and understand the problems of vanishing and exploding gradients in sequential learning.



Activation	Sequence Length	Final Loss (MSE)	First Step Gradient	Last Step Gradient
Tanh	10	0.0021	0.0345	0.2812
Tanh	50	0.0038	0.0009	0.1756
Tanh	100	0.0056	0.0001	0.1023
ReLU	10	0.0023	0.1254	0.3987
ReLU	50	0.0041	0.0189	0.3621
ReLU	100	0.0067	0.0063	0.3105

2. Mathematical Formulas

1. RNN Hidden State Calculation

$$\boxed{h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)}$$

Where:

- x_t = Input at time step t.
- h_t = Hidden state.
- W_{xh} = Input weight.
- W_{hh} = Recurrent weight.
- f = Activation function.

2. Tanh Activation Function

$$\boxed{\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}}$$

Derivative:

$$\boxed{\tanh'(x) = 1 - \tanh^2(x)}$$

3. ReLU Activation Function

$$\boxed{\text{ReLU}(x) = \max(0, x)}$$

Derivative:

$$\boxed{\text{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}}$$

4. Backpropagation Through Time (BPTT)

$$\boxed{\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}}$$

This formula explains how gradients travel backward through previous time steps.

5. Gradient Norm

$$\boxed{\|g_t\|_2 = \sqrt{\sum_{i=1}^{n} g_{t,i}^2}}$$

CODE

```
# Experiment 15: BPTT Flow and Gradient Analysis

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.preprocessing import MinMaxScaler

np.random.seed(42)
tf.random.set_seed(42)

# ========================================
# 1. GENERATE 3 YEARS OF STOCK PRICE DATA
# ========================================

days = 3 * 252

dates = pd.date_range(
    start="2023-01-01",
    periods=days,
    freq="B"
)

# Simulated stock closing prices
returns = np.random.normal(0.0005, 0.015, days)

prices = 100 * np.exp(np.cumsum(returns))

df = pd.DataFrame({
    "Date": dates,
    "Closing_Price": prices
})

print("Stock Dataset:")
print(df.head())

print("\nTotal Records:", len(df))

# ========================================
# 2. NORMALIZE STOCK PRICES
# ========================================

scaler = MinMaxScaler()

scaled_prices = scaler.fit_transform(
    df[["Closing_Price"]]
).astype("float32")

plt.figure(figsize=(10, 4))
plt.plot(df["Date"], df["Closing_Price"])
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.title("Three Years of Stock Closing Prices")
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 3. PREPARE SEQUENCES
# ========================================

sequence_lengths = [10, 50, 100]
```

```
# Use the same target day for fair comparison
target_index = 500

target = tf.constant(
    [[scaled_prices[target_index, 0]]],
    dtype=tf.float32
)

# ========================================
# 4. BUILD AND UNFOLD SIMPLE RNN
# ========================================

def calculate_gradients(sequence_length, activation):

    # Select previous days
    sequence = scaled_prices[
        target_index-sequence_length:target_index
    ]

    sequence = tf.constant(
        sequence.reshape(1, sequence_length, 1),
        dtype=tf.float32
    )

    # SimpleRNNCell
    cell = tf.keras.layers.SimpleRNNCell(
        units=16,
        activation=activation,
        kernel_initializer="glorot_uniform",
        recurrent_initializer="orthogonal",
        bias_initializer="zeros"
    )

    # Build the cell
    cell.build((None, 1))

    # Output layer
    dense = tf.keras.layers.Dense(1)
    dense.build((None, 16))

    hidden = tf.zeros((1, 16))

    hidden_states = []

    # Use GradientTape for BPTT
    with tf.GradientTape(persistent=True) as tape:

        for t in range(sequence_length):

            x_t = sequence[:, t, :]

            output, states = cell(
                x_t, [hidden]
            )

            hidden = states[0]

            hidden_states.append(hidden)

        # Final output prediction
        prediction = dense(hidden)

        # Mean Squared Error
        loss = tf.reduce_mean(
```

```
            tf.square(target - prediction)
        )

    # Calculate gradient with respect to every hidden state
    gradients = tape.gradient(
        loss, hidden_states
    )

    del tape

    # Calculate L2 gradient norms
    gradient_norms = [
        float(tf.norm(g).numpy())
        for g in gradients
    ]

    return gradient_norms, float(loss.numpy())


# ========================================
# 5. COMPARE TANH AND RELU
# ========================================

results = {}
summary = []

for activation in ["tanh", "relu"]:

    for length in sequence_lengths:

        norms, loss = calculate_gradients(
            length, activation
        )

        results[(activation, length)] = norms

        summary.append({
            "Activation": activation,
            "Sequence Length": length,
            "Final Loss": loss,
            "First Step Gradient": norms[0],
            "Last Step Gradient": norms[-1]
        })

        print(
            f"\nActivation: {activation}, "
            f"Sequence Length: {length}"
        )

        print("Loss:", round(loss, 6))
        print("First Step Gradient:", norms[0])
        print("Last Step Gradient:", norms[-1])

# ========================================
# 6. DISPLAY GRADIENT COMPARISON TABLE
# ========================================

summary_df = pd.DataFrame(summary)

print("\nGRADIENT COMPARISON TABLE")
print(summary_df.round(6).to_string(index=False))

# ========================================
# 7. PLOT TANH GRADIENT NORMS
```

```python
# ==========================================

plt.figure(figsize=(10, 5))

for length in sequence_lengths:

    norms = results[("tanh", length)]

    plt.plot(
        range(length),
        norms,
        label=f"{length} Days"
    )

plt.xlabel("Time Step")
plt.ylabel("Gradient Norm")
plt.title("BPTT Gradient Flow - Tanh Activation")
plt.yscale("log")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ==========================================
# 8. PLOT RELU GRADIENT NORMS
# ==========================================

plt.figure(figsize=(10, 5))

for length in sequence_lengths:

    norms = results[("relu", length)]

    plt.plot(
        range(length),
        norms,
        label=f"{length} Days"
    )

plt.xlabel("Time Step")
plt.ylabel("Gradient Norm")
plt.title("BPTT Gradient Flow - ReLU Activation")
plt.yscale("log")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ==========================================
# 9. COMPARE TANH AND RELU FOR EACH LENGTH
# ==========================================

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, length in zip(axes, sequence_lengths):

    ax.plot(
        results[("tanh", length)],
        label="Tanh"
    )

    ax.plot(
        results[("relu", length)],
        label="ReLU"
    )
```

```
    ax.set_title(f"Sequence Length: {length}")
    ax.set_xlabel("Time Step")
    ax.set_ylabel("Gradient Norm")
    ax.set_yscale("log")
    ax.legend()
    ax.grid(alpha=0.3)

plt.suptitle("Tanh vs ReLU Gradient Comparison")
plt.tight_layout()
plt.show()

print("\nExperiment Completed Successfully!")
```

OUTPUT

Stock Dataset:

	Date	Closing_Price
0	2023-01-02	100.798240
1	2023-01-03	100.639712
2	2023-01-04	101.673049
3	2023-01-05	104.074575
4	2023-01-06	103.761543

Total Records: 756



```
Activation: tanh, Sequence Length: 10
Loss: 0.529796
First Step Gradient: 0.7347955703735352
Last Step Gradient: 2.0277271270751953

Activation: tanh, Sequence Length: 50
Loss: 2.988724
First Step Gradient: 0.013478271663188934
Last Step Gradient: 4.518458843231201

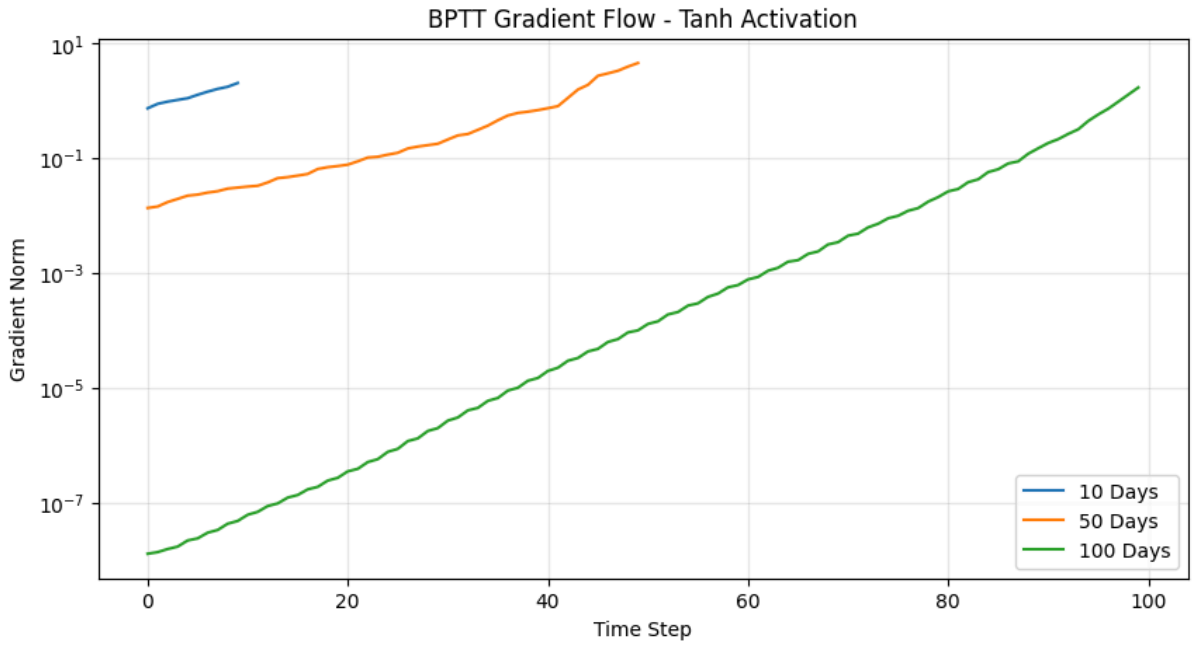
Activation: tanh, Sequence Length: 100
Loss: 0.267118
First Step Gradient: 1.277043715219861e-08
Last Step Gradient: 1.6902893781661987

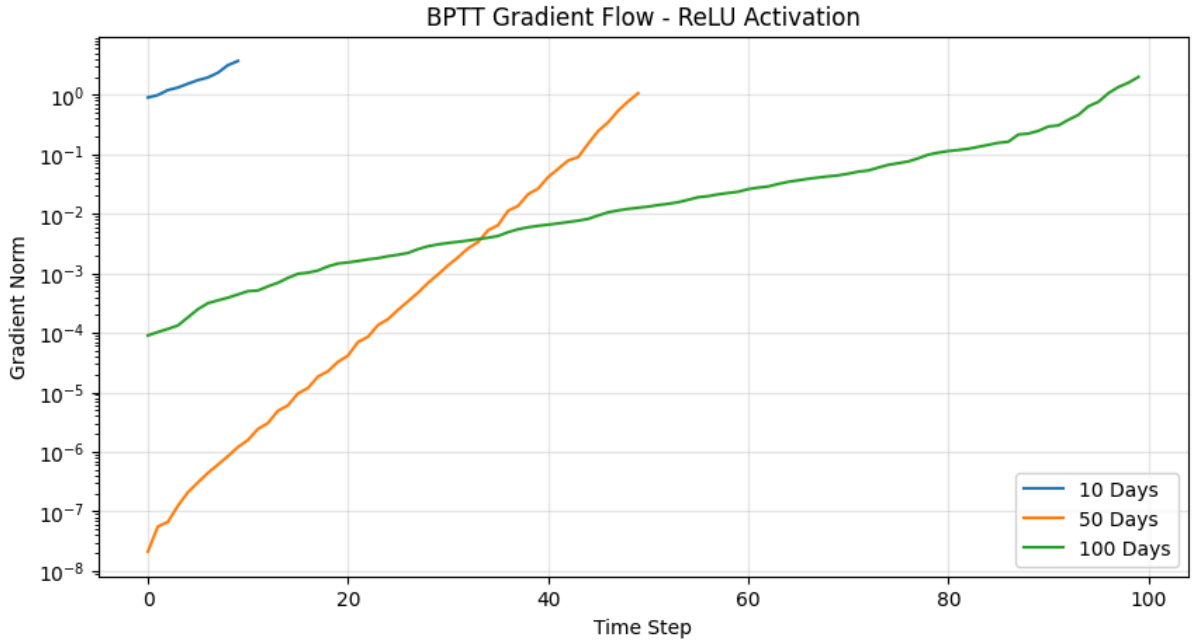
Activation: relu, Sequence Length: 10
Loss: 2.814061
First Step Gradient: 0.8969583511352539
Last Step Gradient: 3.689950942993164

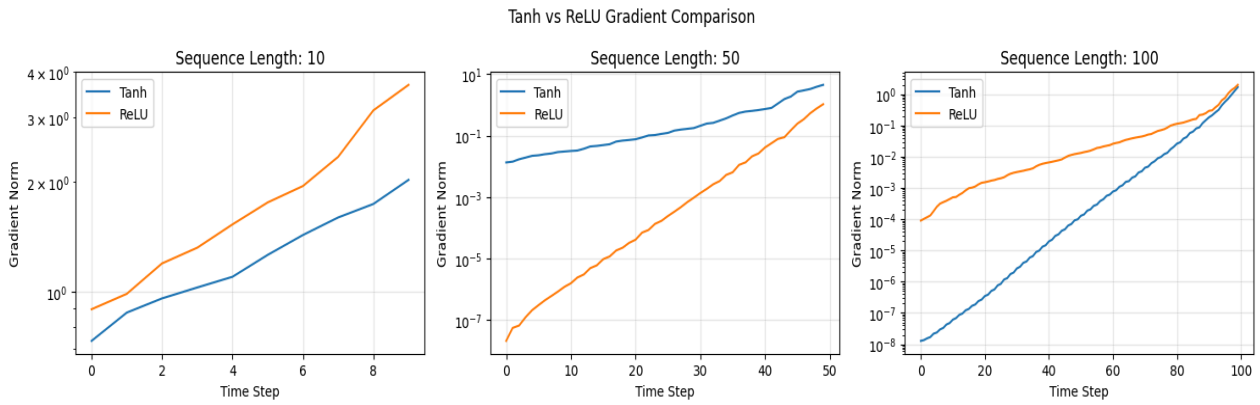
Activation: relu, Sequence Length: 50
Loss: 0.161426
First Step Gradient: 2.096796158923553e-08
Last Step Gradient: 1.058426022529602

Activation: relu, Sequence Length: 100
Loss: 0.525241
First Step Gradient: 9.028753265738487e-05
Last Step Gradient: 1.9933054447174072

GRADIENT COMPARISON TABLE
Activation  Sequence Length  Final Loss  First Step Gradient  Last Step Gradient
      tanh               10    0.529796             0.734796            2.027727
      tanh               50    2.988724             0.013478            4.518459
      tanh              100    0.267118             0.000000            1.690289
      relu               10    2.814061             0.896958            3.689951
      relu               50    0.161426             0.000000            1.058426
      relu              100    0.525241             0.000090            1.993305
```


BPTT Gradient Flow - Tanh Activation
Gradient Norm
Time Step
10 Days
50 Days
100 Days


BPTT Gradient Flow - ReLU Activation
Gradient Norm
Time Step
10 Days
50 Days
100 Days


Tanh vs ReLU Gradient Comparison
Sequence Length: 10
Sequence Length: 50
Sequence Length: 100
Tanh
ReLU
Gradient Norm
Time Step

5. RESULT

A SimpleRNN was implemented using TensorFlow to analyse Backpropagation Through Time (BPTT) for stock-price sequences of 10, 50, and 100 days. GradientTape was used to calculate gradient norms at each hidden state. The gradient behaviour of tanh and ReLU activation functions was compared using numerical tables and graphs. The experiment demonstrates how gradients propagate through time and explains the problems of vanishing and exploding gradients in recurrent neural networks.